

A decorative frame composed of two thick, light-colored L-shaped lines. One line starts at the top-left and extends horizontally to the right, then vertically downwards. The other line starts at the bottom-right and extends horizontally to the left, then vertically upwards. They meet at the center, framing the text.

# ЗАЩИТА БАЗ ДАННЫХ

# Введение

Компании все чаще применяют веб-технологии как основу для оптимизации бизнес-процессов с помощью CRM, ERP и других систем для работы с данными. Это имеет свои преимущества, но и добавляет в информационную систему новые угрозы. Рассмотрим защиту данных в веб-приложениях, основанных на трехуровневой архитектуре, когда пользователь через браузер обращается не напрямую к серверу баз данных, а к серверу приложений, от которого уже идет запрос данных к серверу баз данных.

# Введение

Трехуровневая архитектура имеет большое количество составляющих: рабочие места сотрудников, серверы приложений, СУБД, базы данных, каналы связи. Каждый из этих компонентов представляет собой потенциальный источник утечки информации:

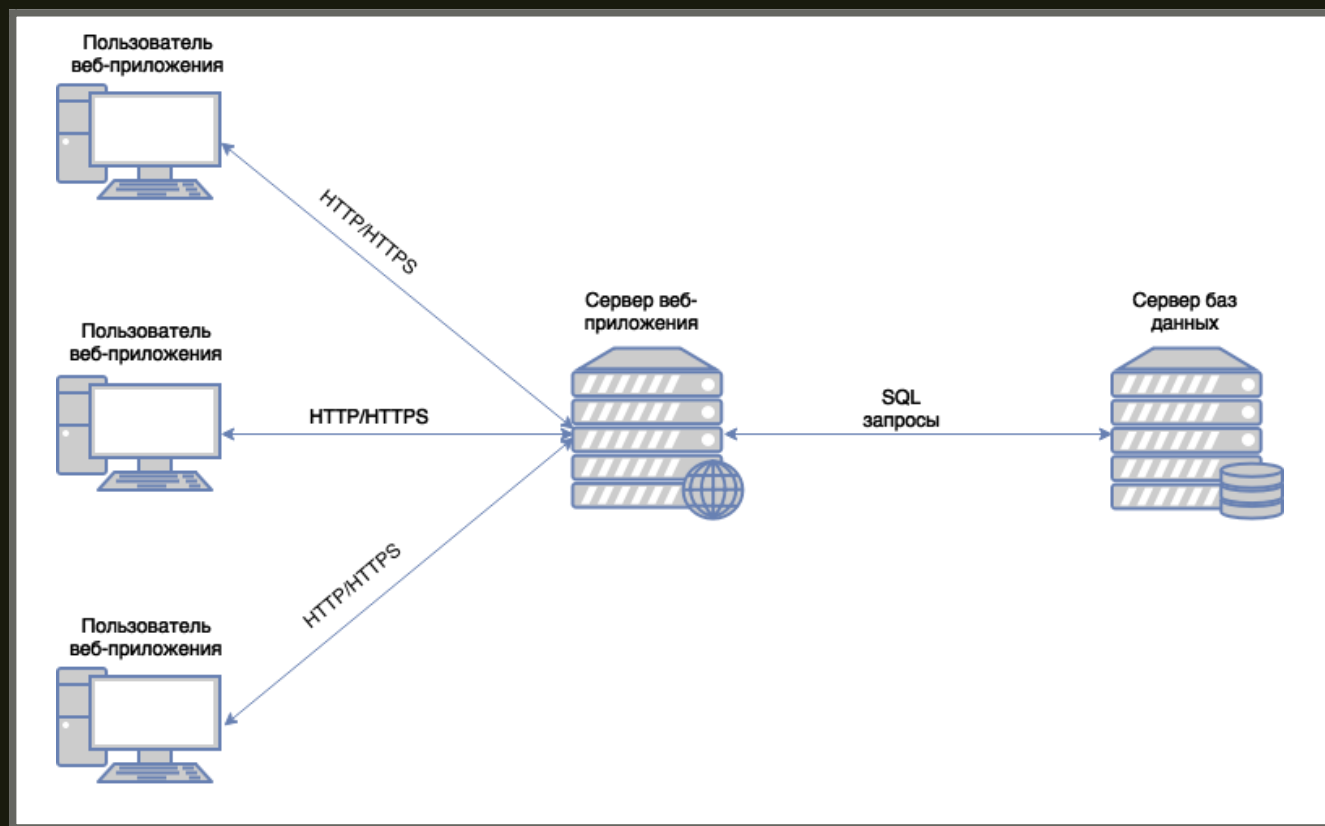
- ☐ перехват трафика до и после сервера веб-приложения;
- ☐ получение информации через уязвимости веб-приложения;
- ☐ хищение информации сотрудниками, имеющими легитимный доступ;
- ☐ выгрузка информации напрямую из базы данных.

# Введение

Однако внедрение всех возможных средств защиты часто не оправдано. Для построения эффективной и достаточной системы защиты необходимо:

- ☐ провести полный аудит защищаемого приложения;
- ☐ установить ценность обрабатываемой информации;
- ☐ составить матрицу доступа пользователей к данным;
- ☐ построить карты всех модулей приложения и их взаимодействия;
- ☐ определить слабые места и самые вероятные точки утечки информации.

# Введение



# Базовые меры повышения уровня защиты веб-приложений

Первый шаг к обеспечению защищенности — правильная настройка всех компонентов системы, обновление всего используемого программного обеспечения до актуальных версий, отключение всех неиспользуемых служб, смена всех паролей по умолчанию, отключение неиспользуемых учетных записей, в том числе системных. Важно также обеспечить шифрование данных всех сетевых соединений внутри системы. В случае если это позволяет веб-приложение, необходимо настроить разграничение прав доступа пользователей к данным и настройкам системы, ограничив их минимально необходимыми.

# Базовые меры повышения уровня защиты веб-приложений

В простых случаях уже только перечисленные базовые меры позволят снизить риски до допустимого уровня. Если базовых мер окажется недостаточно, необходимо обратить внимание на разнообразные специализированные средства защиты информации.

# Аутентификация пользователей как слабое звено веб-приложений

Одно из наиболее известных слабых мест – это аутентификация пользователей веб-приложения. Например, система при регистрации пользователя не проводит проверку на сложность пароля, или пароль не имеет срока действия, или при аутентификации пароль от браузера передается к серверу в URL запросе.



# Аутентификация пользователей как слабое звено веб-приложений

Для устранения подобных проблем существуют решения для усиления аутентификации в веб-приложении, реализованные как в виде встраиваемых модулей, так и в виде отдельных серверов аутентификации.

# Аутентификация пользователей как слабое звено веб-приложений

Встроенная в приложение аутентификация позволяет не использовать внешние средства аутентификации, однако это не исключает возможные ошибки в самом веб-приложении, позволяющие обойти систему аутентификации. Решить эту проблему можно, например, путем внедрения процесса менеджмента уязвимостей.

# Менеджмент уязвимостей

Если в компании используются приложения, разработанные на заказ сторонним подрядчиком, и исходный код приложений недоступен, сотрудникам информационной безопасности при приемке программного обеспечения имеет смысл использовать сканер уязвимостей, который работает с веб-приложением по принципу «черного ящика». При нахождении критических уязвимостей приложение возвращается на доработку разработчику.

# Менеджмент уязвимостей

В случае, когда веб-приложение является внутренней разработкой компании, рекомендуется выстраивать процесс безопасной разработки программного обеспечения с использованием анализаторов исходного кода, проверяя весь разрабатываемый код на отсутствие ошибок, приводящих к уязвимостям. Такой подход позволяет исправить ошибки в приложении на раннем этапе и избежать лишних затрат. Большинство анализаторов кода позволяет проводить как статический анализ кода без его выполнения, так и динамический, проверяя уже установленное и запущенное приложение. В последнем случае требуется указание точек входа и большое количество входных данных.

# Менеджмент уязвимостей

Сочетание перечисленных методов позволяет выявлять уязвимости до внедрения приложений в компании. В то же время реализация в компании процессов верификации приложений может потребовать значительных материальных и временных затрат. Иногда просто нет возможности оперативно вносить изменения в уже работающее приложение. Если ценность данных не позволяет закрыть глаза на возможные уязвимости, имеет смысл рассмотреть возможность использования специализированных средств защиты веб-приложений — Web Application Firewall (WAF). Существуют как коммерческие, так и свободно распространяемые системы.

# Менеджмент уязвимостей

Принцип работы специализированной защиты WAF

Принцип работы WAF: HTTP-трафик от пользователей до веб-приложения проходит сначала через WAF либо, в зависимости от задач и возможностей, на WAF направляется копия трафика. Далее трафик подвергается декодированию и проверке на наличие атак. Если WAF установлен «в разрыв» (прокси, мост), атаки могут быть заблокированы. В пассивном режиме работы (копия трафика) возможны только мониторинг и оповещение об атаках.

# Менеджмент уязвимостей

Для обнаружения атак могут использоваться такие методы, как:

- ☐ сигнатурный анализ;
- ☐ репутационные списки;
- ☐ автоматическое обучение;
- ☐ поведенческий анализ;
- ☐ вручную настраиваемые правила.

# Менеджмент уязвимостей

Кроме этого, WAF может иметь модули для динамического анализа уязвимостей защищаемых приложений, виртуального патчинга найденных уязвимостей, управления аутентификацией пользователей, взаимодействия с другими системами защиты. Все это позволяет снизить количество актуальных для веб-приложения угроз.



# Эффективные способы защиты информации в базах данных

Опасность для информации в веб-приложении представляют и внутренние нарушители — сотрудники, которые имеют доступ к данным для выполнения служебных обязанностей, администраторы с прямым доступом к серверу баз данных (в обход веб-приложения или локально). В этом случае для обеспечения безопасности веб-приложений возможно использовать решения, реализующие разный подход к мониторингу и контролю обращений к базам данных.

# Эффективные способы защиты информации в базах данных

Системы защиты информации в базах данных можно разделить на три типа:

1. Системы, использующие принцип работы, схожий с WAF, — перехват трафика, но идущего не от пользователя до сервера приложений, а от сервера приложений к серверам баз данных. Для контроля локальных и прямых сетевых подключений к базам данных используются агенты, устанавливаемые непосредственно на серверы баз данных. При использовании таких систем для защиты данных в веб-приложениях с трехзвенной архитектурой может возникнуть сложность с определением пользователя, сделавшего запрос к базе данных: в трафике, идущем от сервера приложений, все обращения производятся от служебной учетной записи. Для персонификации сотрудника предусмотрена интеграция с WAF, который анализирует трафик до сервера приложений, либо подача копии этого трафика непосредственно на систему защиты баз данных.

# Эффективные способы защиты информации в базах данных

Также в системах рассматриваемого типа могут быть реализованы возможности, косвенно повышающие защищенность:

- сканирование на уязвимости баз данных;
- обнаружение баз данных;
- маскирование критичной информации, например номеров кредитных карт (при установке «в разрыв»);
- создание матрицы разграничения прав доступа;
- мониторинг изменений, позволяющий вовремя отследить несанкционированное повышение прав пользователя.

# Эффективные способы защиты информации в базах данных

2. В случаях, когда возможность анализа копии трафика отсутствует или необходимо применить маскирование и блокировки, но установить систему защиты «в разрыв» невозможно, **используются решения, основанные на других принципах перехвата обращений пользователей к базам данных.** Такие решения используют в качестве точки съема **агент, устанавливаемый на защищаемый сервер приложения и взаимодействующий с драйверами, через которые веб-приложение передает запросы пользователей к базам данных.** Так как агент находится на сервере веб-приложения, он обрабатывает и запросы клиентов приложению и запросы приложения к базам данных, персонифицируя запросы. Создавая правила обработки запросов, можно маскировать «на лету» любые поля в ответах от базы данных, блокировать нелегитимные запросы, полностью управлять бизнес-процессом работы пользователя с приложением.

# Эффективные способы защиты информации в базах данных

3. Системы защиты информации от утечек, основанные на использовании криптографии и позволяющие выборочно шифровать информацию, хранящуюся в таблицах баз данных. Доступ к информации предоставляется только авторизованным пользователям, с ведением детальных протоколов их действий. Для повышения защищенности информации такие системы могут дополнительно реализовывать механизмы строгой двухфакторной аутентификации.

# Эффективные способы защиты информации в базах данных

Использование перечисленных средств позволит существенно снизить риск утечки данных из веб-приложения, а при инцидентах поможет отыскать необходимую для расследования информацию.

Последний рубеж защиты — правильное хранение резервных копий баз данных: если средства шифрования при работе с базой данных не используются, необходимо шифровать ее резервные копии.

# Выводы

Выбор средств защиты веб-приложений является комплексной задачей и не ограничивается использованием исключительно WAF-решений. Необходимо взвешенно оценивать угрозы, актуальные именно для рассматриваемого веб-приложения, использовать данные аудита защищенности веб-приложений, а также учитывать особенности инфраструктуры и процессов обработки данных.